Conception et programmation orientée objet d'une application web

MaCave • Gestion collaborative d'une cave à vin

Sujet de mini-projet • 4 séances de 4 heures • Application Python exécutée localement

1 Contexte et objectifs

Vous devez concevoir en UML et implémenter en Python une application web multi-utilisateurs permettant à des amateurs de vin de gérer leur cave, de suivre leurs bouteilles et de partager leurs appréciations. Les données doivent être conservées dans une base relationnelle.

Chaque utilisateur dispose d'une cave privée composée d'étagères. Plusieurs exemplaires d'un même vin peuvent être stockés dans une ou plusieurs étagères. Lors d'une dégustation, une bouteille peut être archivée et associée à une note et à un commentaire visible par la communauté.

- Analyser les besoins et formaliser les règles métier.
- Concevoir un modèle objet et des diagrammes UML cohérents avec le code.
- Implémenter des classes Python avec des responsabilités explicites.
- Modéliser et exploiter une base de données relationnelle.
- Développer une interface web avec authentification et contrôle d'accès.
- Tester les règles métier et documenter une application exécutable.

2 Périmètre du projet

Le projet doit être réalisable pendant les 16 heures encadrées, sans nécessiter de travail supplémentaire à domicile.

L'interface doit être lisible et utilisable : formulaires compréhensibles, messages d'erreur explicites, navigation cohérente et libellés associés aux champs. Aucun déploiement sur un serveur externe n'est exigé.

Le code, la documentation et les diagrammes UML seront accessibles sur une forge logicielle telle que GitHub ou GitLab.

3 Utilisateurs et organisation de la cave

3 1 Comptes et confidentialité

- Créer un compte avec un identifiant unique, une adresse électronique et un mot de passe.
- Se connecter, se déconnecter et accéder à son espace personnel.
- Modifier les informations de son profil.
- Consulter et modifier uniquement sa propre cave et son historique privé.

Les mots de passe doivent être stockés sous forme de hachages adaptés, jamais en clair. Les contrôles d'accès doivent être effectués côté serveur, y compris lors d'un accès direct à une URL.

R1 — Un utilisateur ne peut ni consulter le stock privé ni modifier les caves, étagères, bouteilles ou évaluations personnelles d'un autre utilisateur. Les évaluations communautaires restent consultables par tous les utilisateurs authentifiés.

3 2 Cave et étagères

Chaque utilisateur dispose d'une cave virtuelle. Il peut créer, modifier et supprimer ses étagères, consulter leur contenu et connaître leur taux d'occupation.

Attribut d'une étagère	Description
Identifiant	Identifiant unique
Nom	Nom choisi par le propriétaire
Capacité maximale	Nombre maximal de bouteilles physiques
Occupation	Nombre de bouteilles actuellement en stock

- R2 — Les noms des étagères sont uniques au sein d'une même cave.
- R3 — L'occupation ne peut jamais dépasser la capacité maximale.
- R4 — La capacité ne peut pas être réduite en dessous de l'occupation actuelle.
- R5 — Une étagère non vide ne peut être supprimée avant le déplacement ou le retrait de ses bouteilles.

La capacité et les quantités saisies sont des entiers strictement positifs. Une opération refusée ne doit laisser aucune modification partielle en base.

3 3 Données privées et données partagées

Le stock, les emplacements, les prix d'achat et l'historique personnel sont privés. Le catalogue de vins ainsi que les notes et commentaires de dégustation sont partagés. La publication d'une évaluation ne doit pas révéler le contenu de la cave de son auteur.

4 Vins bouteilles et dégustations

4 1 Distinguer une référence de ses exemplaires

Un vin représente une référence commune du catalogue. Une bouteille représente un exemplaire physique détenu par un utilisateur. Cette distinction doit apparaître dans la conception et dans le code : cinq bouteilles identiques correspondent à cinq exemplaires rattachés à une seule référence.

Données de la référence de vin	Exemple
Domaine viticole	Château Montelena
Nom ou cuvée	Cabernet Sauvignon
Type	Rouge, blanc, rosé ou effervescent
Millésime	1967
Région	Californie
Description	Présentation générale du vin
Photo de l'étiquette	Image associée à la référence
Note Communautaire	Moyenne calculée à partir des évaluations

L'identité d'une référence repose sur le domaine, le nom ou la cuvée, le type, le millésime et la région. Le prix d'achat et les commentaires ne définissent pas cette identité. Les étudiants expliqueront comment ils évitent les doublons, notamment ceux dus aux espaces et à la casse.

Données d'une Bouteille	Description
Identifiant et vin associé	Identifiant unique et lien vers la référence
Propriétaire et emplacement	Cave et étagère de stockage
Prix d'achat et date d'ajout	Prix propre à cet exemplaire et date d'entrée
Statut	En stock ou archivée

4 2 Gestion du stock

- Créer une référence ou sélectionner une référence existante.
- Ajouter une ou plusieurs bouteilles identiques et modifier leurs informations autorisées.
- Consulter les fiches des vins et les exemplaires détenus.
- Déplacer une bouteille vers une autre étagère de sa cave.
- Retirer définitivement ou archiver un exemplaire.

R6 — Le retrait d'une bouteille parmi cinq exemplaires laisse quatre bouteilles en stock. R7 — Tout déplacement respecte la capacité de destination. L'ajout de plusieurs exemplaires et le déplacement doivent être atomiques : toute l'opération réussit ou aucune modification n'est conservée.

4 Vins bouteilles et dégustations suite

4 3 Retrait et archivage

Retrait définitif : la bouteille est supprimée de l'inventaire personnel ; la référence du vin reste dans le catalogue communautaire.

Archivage après dégustation : la bouteille quitte le stock actif et reste dans l'historique personnel avec une date de sortie, une note comprise entre 0 et 20 et un commentaire facultatif. Une bouteille archivée n'occupe plus de place sur une étagère.

R8 — L'archivage n'affecte aucun autre exemplaire du même vin. L'application doit empêcher qu'une bouteille déjà archivée soit retirée ou archivée une seconde fois.

4 4 Notes et commentaires communautaires

Chaque archivage permet d'enregistrer une évaluation liée à la bouteille dégustée. Les utilisateurs authentifiés peuvent consulter les évaluations associées à une même référence de vin.

- R9 — Seules les notes effectivement enregistrées entrent dans le calcul.
- R10 — Plusieurs dégustations du même vin peuvent produire plusieurs évaluations, y compris par un même utilisateur. Chaque dégustation a le même poids dans la moyenne.
- R11 — Chaque évaluation est associée à un auteur, à un vin et à une bouteille archivée.
- R12 — Un utilisateur peut modifier uniquement ses propres évaluations.

La note moyenne est la somme des notes divisée par leur nombre. Ainsi, les notes 16, 12 et 17 donnent une moyenne de 15/20. Sans évaluation, afficher « Non évalué », et non zéro. Toute modification d'une note doit être répercutée dans la moyenne.

5 Consultation et tableau de bord

5 1 Recherche filtrage et tri

- Afficher les bouteilles en stock, le contenu d'une étagère et l'historique des dégustations.
- Rechercher un vin par son nom ou son domaine et filtrer par type.
- Trier dans les deux sens par domaine, nom, type, millésime, région, prix d'achat et note communautaire.
- Permettre le tri de l'historique par note personnelle. Préciser le traitement des valeurs absentes.

Un affichage regroupé par référence est autorisé s'il indique la quantité et permet d'accéder aux exemplaires individuels. Si plusieurs prix existent, le tri par prix doit porter sur les exemplaires ou appliquer une convention explicitée.

5 2 Indicateurs personnels

Le tableau de bord affiche le nombre de bouteilles en stock, le nombre de références distinctes, le nombre d'étagères, l'occupation globale et la valeur du stock calculée à partir des prix d'achat. L'occupation globale correspond au nombre de bouteilles divisé par la somme des capacités. Sans étagère, afficher un état explicite, sans division par zéro.

6 Contraintes techniques et conception

6 1 Technologies recommandées

Composant	Technologie
Langage et framework web	Python 3 et Flask
Base et accès aux données	SQLite
Interface	Jinja2, HTML5, CSS3 ; Bootstrap facultatif
Modélisation et versions	UML et Git sur GitHub ou GitLab
Tests	pytest ou unittest

Un autre framework Python peut être utilisé si le choix est justifié. L'usage d'un ORM ne dispense pas de comprendre et de présenter le modèle relationnel.

6 2 Modèle objet et architecture

Le modèle métier doit comporter des classes Python explicites. Les concepts Utilisateur, Cave, Etagere, Vin, Bouteille et Evaluation constituent des pistes ; les responsabilités, méthodes, associations et choix de représentation doivent être justifiés.

Les classes ne doivent pas être uniquement des structures de données. La vérification des capacités, les déplacements, l'archivage et les calculs doivent être portés par des méthodes ou services métier testables indépendamment des requêtes HTTP.

- Interface : pages HTML, formulaires et templates.
- Couche web : routes, sessions, validation des requêtes et réponses HTTP.
- Couche métier : classes et services qui appliquent les règles de gestion.
- Persistance : relations, contraintes, requêtes et transactions.

Une architecture simple et modulaire est attendue. Il n'est pas nécessaire de multiplier les couches ni les abstractions. Les entrées doivent être validées côté serveur et les opérations sensibles doivent préserver l'intégrité des données.

6 3 Diagrammes UML attendus

- Diagramme de cas d'utilisation présentant les interactions principales.
- Diagramme de classes avec attributs typés, méthodes principales, associations et multiplicités ; composition ou agrégation si pertinente.
- Au moins un diagramme de séquence : ajout multiple, déplacement ou archivage avec évaluation.

Les diagrammes doivent être cohérents avec l'implémentation finale et peuvent évoluer pendant le projet. Le modèle doit rendre explicite la distinction entre une référence de vin et ses exemplaires physiques.

7 Validation et organisation

7 1 Scénarios fonctionnels de référence

Scénario	Résultat attendu
Créer un compte	Connexion possible et cave privée accessible
Créer une étagère de capacité 10	Occupation initiale de 0/10
Ajouter 5 bouteilles identiques	5 exemplaires et une seule référence
Ajouter 6 bouteilles à une étagère occupée à 5/10	Refus sans ajout partiel
Déplacer une Bouteille	Occupation des deux étagères actualisée
Archiver un exemplaire parmi 5	4 en stock et 1 dans l'historique
Enregistrer les notes 16, 12 et 17	Moyenne communautaire de 15/20
Se connecter avec un second compte	Stock privé du premier inaccessible
Consulter une référence partagée	Évaluations visibles pour les deux comptes
Supprimer une étagère non vide	Refus et message explicite

Prévoir au moins cinq tests automatisés couvrant les règles métier : dépassement de capacité, retrait d'un exemplaire, déplacement, archivage et moyenne communautaire. Compléter par des vérifications de contrôle d'accès, de données invalides et de persistance après redémarrage.

8 Livrables attendus

Mettre à disposition un dépôt sur une forge logicielle contenant :

- Le code source complet et les dépendances nécessaires (requirements.txt ou équivalent).
- Un README précisant l'installation, l'initialisation de la base, le lancement et l'exécution des tests.
- Les diagrammes UML à jour, de préférence avec leurs sources éditables.
- Une description succincte des choix de conception, du modèle relationnel et des conventions retenues.
- Les tests automatisés et un jeu de démonstration : au moins deux utilisateurs, plusieurs étagères et vins, des exemplaires identiques et des évaluations.

L'application doit fonctionner localement sur les postes de TP. Le dépôt ne doit pas contenir de secrets. Les éventuelles limites ou fonctionnalités inachevées doivent être indiquées dans le README.

9 Évaluation

L’évaluation repose sur une revue du code, l’analyse des documents UML et une démonstration fonctionnelle lors de la séance d’examen. Date et horaire à confirmer par l’enseignant.

Critère	Points
Analyse et conception UML	4
Programmation orientée objet et architecture	4
Utilisateurs, étagères et stocks	3
Bouteilles, retrait, archivage et évaluations	3
Base de données, intégrité et persistance	2
Interface, ergonomie, recherche et tri	2
Tests, documentation et qualité du dépôt Git	2
Total	20

Le barème est indicatif. La robustesse, la cohérence entre UML et code, la qualité des responsabilités et la capacité à expliquer les choix comptent autant que la présence des fonctionnalités. Chaque membre du groupe doit pouvoir expliquer les principales parties de l’application.